

Bluepill STM32F103 — UART1 DMA (circular) + PCA9685 Servo Controller

****Pins (this build):****

- ****UART1****: PA9 (TX), PA10 (RX) — Protocol link to host (Node-RED / Pi). RX uses ****DMA circular**** + ****IDLE**** IRQ.
- ****UART2****: PA2 (TX), PA3 (RX) — Human-readable console / debug.
- ****I2C1****: PB6 (SCL), PB7 (SDA) — PCA9685 at 0x40 (HAL 8-bit address 0x80). External 2.2–4.7 kΩ pull-ups to 3V3 recommended.

****Key features:****

- Framed protocol: `A` (angles), `V` (per-finger ms/deg), `G` (global ms/deg), `N` (motor count), `MODE`, `LOOP`.
- Non-blocking per-degree motion planner with I2C round-robin throttle.
- Watchdogs: RX silence → SAFE pose (AUTO only). I2C stall → auto bus-clear + PCA re-sync.
- Deterministic completion reporting: `ELAPSED`, `DONE` on UART1.

Boot & Init

1. `HAL\_Init()` + `SystemClock\_Config()`
2. Peripherals: `MX\_GPIO\_Init()`, `MX\_DMA\_Init()`, `MX\_USART2\_UART\_Init()`, `MX\_USART1\_UART\_Init()`, `MX\_I2C1\_Init()`
3. ****UART1 RX****:
 - `U1RX\_Init(&huart1;)` configures ****circular**** RX DMA ring.
 - `U1RX\_Start()` enables DMA + IDLE IRQ.
 - In `stm32f1xx\_it.c`, the ISR ****must**** clear IDLE (and ORE) before calling `U1RX\_OnUsartIrq(&huart1;)`.
4. ****PCA9685****: `PCA9685\_Init(50)` → MODE2 OUTDRV, MODE1 AI, prescale set via rounded 25 MHz/4096 formula.
5. ****Motors****: `MF\_GetMotorCount()` (default 8) → clamp ≤ 16 → `g\_servo\_id[i] = i`.
6. ****Motion tables****: `Motion\_Init(N)` sets SAFE pose and schedules first steps. Console banner on UART2.

Main Loop (each tick)

1. ****handle_meta_updates****
 - Applies N, G, MODE, LOOP if present in `MF\_GetMetaUpdates()`; ACK via `DONE` lines on UART1.
 - Touches `s\_last\_cmd\_ms` (feeds RX watchdog).
2. ****apply_targets_from_parser****
 - For each bit in `MF\_GetUpdatedMask()`, update target angle (clamped) and latch per-motor speed when `V>0`.
 - Remember last full `A` in `g\_last\_cmd\_angles[]` (for AUTO ping-pong).
 - Start timing for the move: `s\_step\_active=1`, `s\_move\_start\_ms=now`.
3. ****Motion_Update****
 - Round-robin, at most ****I2C_WRITES_PER_LOOP**** (e.g., 6) writes per loop.
 - Step each affected channel by ****±1**** when its `next\_step\_ms` expires.
 - If `HAL\_I2C\_GetState(&hi2c1;) != READY`, back-off a few ms (don't pile up writes).
4. ****Progress watchdog**** (I2C self-heal)
 - Track sum of `current` angles. If unchanged for `STALL\_MS` (e.g., 800ms) while `s\_step\_active`:
 - ****Bus-clear**** PB6/PB7 (9 SCL pulses until SDA released).

- `PCA9685\_Init(50)` then **re-send** all current angles, set `next\_step\_ms = now`.
- Emit `ERR:I2CSTALL` (UART1) and a console note (UART2).
- Bump `s\_last\_cmd\_ms = now` to avoid RX watchdog during recovery.

5. **Completion**

- When all channels reached:
 - Emit `ELAPSED:` and `DONE` on UART1.
 - If `MODE=AUTO` and `g\_have\_last\_cmd`, ping-pong to SAFE↔last A:.

6. **RX watchdog** (AUTO only)

- If `(now - s\_last\_cmd\_ms) > WATCHDOG\_MS` (e.g., 800ms) → glide to SAFE at default speed and emit `ERR:WATCHDOG` (UART1).

Host Protocol Cheat-Sheet (UART1)

Outbound (device → host):

...

ELAPSED:

DONE

DONE:MODE=AUTO|MANUAL

DONE:LOOP=0|1

DONE:G=

DONE:N=

ERR:I2CSTALL

ERR:WATCHDOG

...

Inbound (host → device) — examples:

...

MODE:AUTO

LOOP:1

N:6

G:20

A:90,45,30,0,0,120

V:30,30,0,0,0,10

A+V: 45/20, 60/10, 0, 0, 0, 90/15

...

- `A:θ1,θ2,...` angles in degrees (0..180).
- `V:ms1,ms2,...` speed per degree (ms/deg); `0` = ignore for that motor.
- `A+V:` pairs `θ/ms`. Missing/short lists are ignored beyond provided items.

Configuration Knobs

- `I2C\_WRITES\_PER\_LOOP` — 6–8 typical.
- `DEFAULT\_STEP\_DELAY\_MS` — default speed (ms/deg).
- `WATCHDOG\_MS`, `STALL\_MS` — comms and I²C stall thresholds.
- `SAFE\_POSE[]` — your neutral angles at boot/failsafe.
- In `PCA9685.h`: `PCA\_SERVO\_MIN\_US`, `PCA\_SERVO\_MAX\_US` (e.g., 1000–2000 μs) to set travel.

ISR Wiring (must-have)

```

c
void USART1_IRQHandler(void)
{
    if ((__HAL_UART_GET_FLAG(&huart1, UART_FLAG_IDLE) != RESET) &&
        (__HAL_UART_GET_IT_SOURCE(&huart1, UART_IT_IDLE) != RESET))
    {
        __HAL_UART_CLEAR_IDLEFLAG(&huart1);
        __HAL_UART_CLEAR_OREFLAG(&huart1);
        U1RX_OnUsartIrq(&huart1);
    }
    HAL_UART_IRQHandler(&huart1);
}

```

Appendix A — Globals & Structs

Symbol	Type	Description
---	---	---
`g_motor_count`	`uint8_t`	Active channels (1..16), default 8.
`g_servo_id[i]`	`uint16_t`	Logical motor → PCA channel (0..15).
`SAFE_POSE[i]`	`uint16_t`	Safe/neutral angles.
`g_finger[i].angle`	`uint16_t`	Requested angle (0..180).
`g_finger[i].speed_ms_per_deg`	`uint16_t`	Latched per-motor speed (ms/deg) when `V>0`.
`g_motion[i].current`	`uint16_t`	Planner angle.
`g_motion[i].target`	`uint16_t`	Target angle.
`g_motion[i].step_delay_ms`	`uint16_t`	Delay per degree step (ms).
`g_motion[i].next_step_ms`	`uint32_t`	Next allowed step time (tick).
`g_speed_latched[i]`	`uint8_t`	1 if per-motor speed was set.
`g_last_cmd_angles[i]`	`uint16_t`	Last full A: set (for AUTO ping-pong).
`g_have_last_cmd`	`uint8_t`	Whether last A: set exists.
`g_auto_flip`	`uint8_t`	Toggles SAFE↔last A: in AUTO.
`s_last_tx_ms`	`uint32_t`	UART2 snapshot throttle timestamp.
`s_last_cmd_ms`	`uint32_t`	Last host activity; used by RX watchdog.
`s_move_start_ms`	`uint32_t`	Move start timestamp (for `ELAPSED:`).
`s_step_active`	`uint8_t`	Motion in progress.
`s_in_failsafe`	`uint8_t`	RX watchdog SAFE pose engaged.

```

**Structs**
```c
typedef struct {
 uint16_t angle;
 uint16_t speed_ms_per_deg;
} FingerData;

typedef struct {
 uint16_t current;
 uint16_t target;
 uint16_t step_delay_ms;
 uint32_t next_step_ms;
} FingerMotion;
```

```

Appendix B — Functions

| Function | Purpose |
|--|--|
| --- | --- |
| `U1RX_Init`, `U1RX_Start`, `U1RX_OnUsartIrq` | UART1 DMA circular RX + IDLE ingestion into parser. |
| | |

| ``handle_meta_updates()`` | Apply N/G/MODE/LOOP; emit ``DONE:*``; touch watchdog. |
 | ``apply_targets_from_parser(mask, count, touched)`` | Update per-motor targets & optionally per-motor speeds; remember last A:. |
 | ``Motion_Init(count)`` | Initialize motion tables to SAFE; send initial PCA positions. |
 | ``Motion_Update(count)`` | Non-blocking $\pm 1^\circ$ stepping with I²C quota & state guard. |
 | ``all_reached(count)`` | True when every ``current == target``. |
 | ``send_snapshot_uart2()`` | Console summary of targets/speeds (human-readable). |
 | ``I2C1_BusClear()`` | Free SDA-low by toggling SCL (PB6/PB7), then re-init I²C. |
 | ``PCA9685_Init(f)`` | Configure PCA9685: OUTDRV, AI, prescale for ``f`` Hz. |
 | ``PCA9685_SetServoAngle(ch, deg)`` | Map 0.. 180° to μs (min/max) $\rightarrow$ 12-bit counts; write LEDx. |

Appendix C — Constants

| Macro | Meaning |
 |---|---|
 | ``MAX_MOTORS`` | Compile-time cap (16). |
 | ``DEFAULT_STEP_DELAY_MS`` | Default ms/degree when no per-motor or global speed. |
 | ``I2C_WRITES_PER_LOOP`` | Max PCA writes per loop (I²C throttle). |
 | ``WATCHDOG_MS`` | RX silence $\rightarrow$ SAFE pose (AUTO only). |
 | ``STALL_MS`` | No progress while moving $\rightarrow$ I²C bus-clear + re-sync. |
 | ``SERVO_MIN_ANGLE``, ``SERVO_MAX_ANGLE`` | Angle clamp (0.. 180°). |
 | ``PCA_SERVO_MIN_US``, ``PCA_SERVO_MAX_US`` | PWM endpoints (μs), e.g., 1000–2000. |
 | ``TX_THROTTLE_MS`` | UART2 snapshot throttle interval. |

Appendix D — Electrical Notes

- External pull-ups 2.2–4.7 k Ω to 3V3 on PB6/PB7.
- $\geq 470 \mu\text{F}$ electrolytic + 0.1 μF ceramic across servo V+ to GND near the PCA board.
- Common ground between Bluepill and the servo power supply.
- PCA9685 `**OE**` held low (enabled).